User Guide

Ordinal-Probit Latent-Class Trajectory Model

File: ordinal_probit.sas Traj2 (released core) **Authors:** Haiqun Lin, Anum Zafar **Updated:** 14 October 2026

What it does

This file fits a class-specific cumulative-probit trajectory model for a bounded ordinal outcome, such as an ADL score, a frailty grade, or a cognitive screening category, by maximum likelihood in PROC NLMIXED. It sorts subjects into a few latent classes, each with its own trend over time, and each class gets its own ordered thresholds. Everything is macro-only with no compiled code, so it runs as-is in the CMS VRDC and any SAS 9.4 or newer session. Including the file defines the macros and runs nothing.

What you need

- SAS 9.4 or newer with SAS/STAT (PROC NLMIXED).
- ODS Graphics for the plots (`%ordprob_mix_plot_one`). The fit itself runs without them.
- A wide, one-row-per-subject dataset, and write access to WORK.

The model in brief

For an outcome with m ordered categories, each class c has a latent index that is a polynomial in time, and its own thresholds. The probability of each category comes from differences of the normal CDF:

```
eta_c(t) = beta_c0 + beta_c1*t + ... + beta_cd*t^d

thresholds (first fixed at 0, then cumulative increments):
  th1_c = 0                  fixed, not estimated
  thj_c = th(j-1)_c + i_cj   j = 2 .. m-1

P(Y=1 | c) = probnorm(th1 - eta)
P(Y=j | c) = probnorm(thj - eta) - probnorm(th(j-1) - eta)
P(Y=m | c) = 1 - probnorm(th(m-1) - eta)

class share:  pie_c = exp(alpha0_c) / sum_k exp(alpha0_k)
```

For identification the class A mixing logit is fixed to 0, the first threshold is fixed at 0, and the class intercept `beta_c0` is free. Ordering of the categories is enforced by a `BOUNDS` statement that the fitting macro generates, holding every increment strictly positive (`i_c2 > 0` , `i_c3 > 0` , and so on). The increments are not exponentiated. Missing time points are allowed and skipped under missing-at-random.

Quick start (simulated data)

The simulator, the base build, the fit, the posterior step, and the plot run as five calls. The simulator's cap sets the highest category, so use `cap = m - 1` to match your category count.

```
/* 1. Simulate a 4-category outcome (cap=3 gives values 0..3) */
%sim_data(dist=BIN4, class=3, n=2000, T=12, seed=1, cap=3);

/* 2. Build BASE_FILE_SRS (adds quar1..quar12 = 1..12, id -> BENE_ID) */
%build_base_from_simwide(T=12);

/* 3. Fit one K=3, quadratic model */
%ordprob_mix_fit_one(
  data=BASE_FILE_SRS, id=BENE_ID,
  yvars=Y1_1-Y1_12, tvars=quar1-quar12, ttotal=12,
  m=4, ycodes=0 1 2 3, deg=2, k=3, prefix=ordprob_T1C3);

/* 4. Posterior class membership, class sizes, APP and OCC */
%ordprob_mix_post_one(
  data=BASE_FILE_SRS, id=BENE_ID,
  yvars=Y1_1-Y1_12, tvars=quar1-quar12, ttotal=12,
  m=4, ycodes=0 1 2 3, deg=2, k=3,
  outestlib=work, prefix=ordprob_T1C3);

/* 5. Plot class proportions + predicted trajectories */
%ordprob_mix_plot_one(outestlib=work, prefix=ordprob_T1C3,
  k=3, ttotal=12, m=4, ycodes=0 1 2 3);
```

Step 4 must use the same `prefix` and `k` as step 3, or it cannot find the estimates.

The settings you edit

ARGUMENT	MEANING
`data / id`	Input wide table and subject id (defaults BASE_FILE_SRS, BENE_ID).
`yvars`	Ordinal outcome columns, one per time point (e.g. Y1_1-Y1_12).
`tvars`	Time-index columns holding values 1..T (e.g. quar1-quar12).
`ttotal`	Number of time points.
`m / ycodes`	Number of categories and their codes (e.g. m=4, 0 1 2 3).
`deg`	Trend shape: 1 linear, 2 quadratic, 3 cubic.
`k`	Number of latent classes.
`start_values`	Optional starting values (see below). Blank uses the defaults.
`bounds`	Optional extra parameter bounds, added to the generated increment bounds.
`prefix`	Prefix for the output dataset names.

Using your own data

Point `data=` at a wide table with one row per subject, or build BASE_FILE_SRS yourself. It must contain the columns named in your arguments.

COLUMN	WHAT TO SUPPLY
`Subject id`	Named by `id`, unique per subject.
`Outcome`	Named by `yvars`, one integer-coded column per time point.
`Time index`	`tvars` (quar1-quarT) holding the values 1..T. Create these yourself.
`Categories`	Set `m` and `ycodes` to match your outcome's codes exactly.

Two things to get right:

- **Codes must match.** A value that is not in `ycodes` is not counted at that time point, so align `ycodes` and `m` with your data.
- **Time runs 1 to T.** `%build_base_from_simwide` fills `quar1-quarT` with 1..T, and the plot axis runs 1..T. Build your own time index the same way.

Running and choosing the number of classes

Each call to `%ordprob_mix_fit_one` fits one model for a fixed `k`. To choose the number of classes, refit with different `k` values and compare BIC in the fit-statistics dataset, preferring the lowest BIC with sensible class sizes and clear separation.

Starting values

Pass name=value pairs in `start_values` and the macro merges them with the defaults into one PARMS statement with no duplicates. Supply the threshold **increments**, from `i*2` upward. There is no `i*1`: the first threshold is fixed at 0. Never supply `alpha0_A`, which is fixed at 0 for identification.

```
%let STARTS = %str(alpha0_B=-0.5 alpha0_C=-0.2
                   beta_a0=0.1 ib2=1 ib3=1);
%ordprob_mix_fit_one(... start_values=&STARTS ...);
```

> **Do not start an increment at 0.** The increments are bounded below by zero, so a start of 0 sits exactly on the boundary. Left blank, the macro starts every increment at **1**, which places the thresholds at 0, 1, 2 and so on. Mixing intercepts and polynomial coefficients do start at 0.

What you get back

- `{prefix}_EST_K{k}`: parameter estimates and standard errors (alpha, beta, threshold increments).
- `{prefix}_FIT_K{k}`: fit statistics (AIC, BIC, -2 log L) for comparing candidate k.
- `{prefix}_ESTS_K{k}`: thresholds on the natural scale, threshold2 upward. threshold1 is fixed at 0 and is not reported.
- `{prefix}_POST_K{k}`: one row per subject with post1..postK, the largest posterior, and the modal class.
- `{prefix}_CLASS_K{k}`: one row per class with n, percent, estimated proportion, APP and OCC.
- Plots from `%ordprob_mix_plot_one`: a class-proportion barplot and predicted E[Y_t | class] trajectory curves.

Checking the classes

`%ordprob_mix_post_one` prints and stores two standard diagnostics per class. APP is the mean posterior probability among the subjects assigned to that class. OCC compares that against what chance alone would give:

```
OCC_j = [APP_j / (1 - APP_j)] / [pie_j / (1 - pie_j)]
```

Nagin's guidance is APP of at least 0.7 and OCC of at least 5 for every class. A class that fails both is usually a sign of too many classes.

Troubleshooting

SYMPTOM	FIX
No APP or OCC anywhere	The fit does not produce them. Run `%ordprob_mix_post_one` after it, with the same `prefix` and `k`.
Values seem ignored, odd likelihood	Outcome codes do not match `ycodes`. Set `m` and `ycodes` to your data's actual codes.
Trajectory or time axis looks off	`tvars` not filled with 1..T. Rebuild `quar` with values 1..T.

SYMPTOM	FIX
Simulated data has an extra category	`cap` does not match `m`. Use `cap = m - 1` in `%sim_data`.
Optimizer stops on the boundary	An increment was started at 0. Start increments at a positive value, or leave `start_values` blank.
Will not converge	Try fewer classes or a lower `deg`, provide `start_values`, or add `bounds` for stability.

CHAO Lab, Rutgers. Full parameter names and macro details are in the data dictionary for ordinal_probit.sas.